

Factoring Learned Clauses

Florian Pollitt 

University Freiburg, Freiburg, Germany

Zachary Battleman 

Carnegie Mellon University, Pittsburgh, Pennsylvania, US

Mathias Fleury 

University Freiburg, Freiburg, Germany

Yakir Vizel 

Technion, Israel Institute of Technology, Israel

Marijn J. H. Heule 

Carnegie Mellon University, Pittsburgh, Pennsylvania, US

Armin Biere 

University Freiburg, Freiburg, Germany

Randal E. Bryant 

Carnegie Mellon University, Pittsburgh, Pennsylvania, US

1 Introduction

One of the greatest strengths of the conflict-driven clause learning (CDCL) paradigm [7] is its flexibility and extensibility. It accommodates a wide range of optimizations and heuristics during search [1, 8, 9]. This is perhaps best exemplified by the development of “inprocessing” [5]—algorithms for the simplification and transformation of formulas that exploit not only the structure of the original formula, but also information learned throughout the solver’s execution. For background on SAT, proof systems, and inprocessing, we refer the reader to the *Handbook of Satisfiability* [2].

In this paper we introduce CADICAL-FX, an extension of bounded variable addition (BVA) [6] with advanced XOR and ITE gate detection. Unlike BVA, CADICAL-FX crucially applies gate detection to *learned* clauses as well (Sect. 2). This extension yields significant improvements in solver performance on Tseitin formulas.

In analyzing the proofs produced by CADICAL-FX, we developed a formalism capable of generating polynomial-length proofs for Tseitin formulas via XOR gate factorization combined with bounded variable elimination [4] in *preprocessing* (Sect. 3). Achieving this requires tuning solver heuristics and a shared bound between these two techniques.

2 Advanced Gate Factorization

The key to advanced gate factorization is to observe changes to the CNF representation of gates in a single variable elimination step [3] (*cf.* Example 1). Doing such an **elimination step in reverse** corresponds to *factoring out* a common pattern from the CNF and can be done by introducing the missing gate definitions with extended resolution.

► **Example 1.** Let us assume that the CNF formula originally contained an XOR gate $x = a \oplus b$ with clausal encoding $(\bar{x} \vee a \vee b), (\bar{x} \vee \bar{a} \vee \bar{b}), (x \vee \bar{a} \vee b), (x \vee a \vee \bar{b})$, as well as clauses $C \vee x$ and $D \vee \bar{x}$. Since both x and $\bar{x}$ occur twice in the XOR gate representation, eliminating x results in two different clause pairs: Resolving with $C \vee x$ yields $(C \vee a \vee b), (C \vee \bar{a} \vee \bar{b})$ while resolving with $D \vee \bar{x}$ yields $(D \vee \bar{a} \vee b), (D \vee a \vee \bar{b})$.



© Florian Pollitt and Zachary Battleman and Mathias Fleury and Yakir Vizel and Marijn J. H. Heule and Armin Biere and Randal E. Bryant;

licensed under Creative Commons License CC-BY 4.0



Leibniz International Proceedings in Informatics

LIPIC Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

3 Polynomial Extended Resolution Proofs of Tseitin Formulas

Our preprocessing approach relies on eliminating cycles for the underlying graph of the Tseitin formula (cf. Fig. 1). By repeating the cycle elimination, we can reduce the graph to a single node, which corresponds to the empty clause. We can achieve cycle elimination without consideration of the graph structure by implementing random tie breakers for factorization.

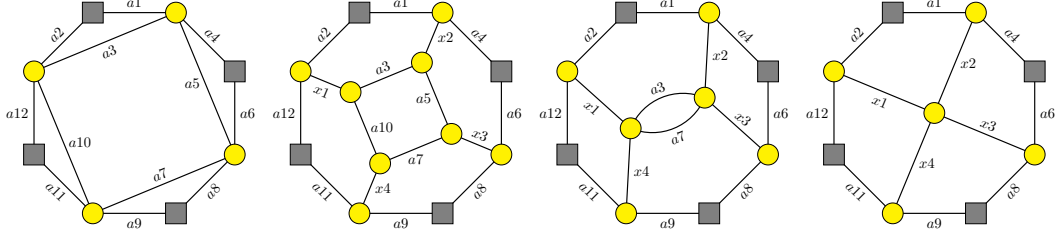


Figure 1 The plot visualizes the elimination of a *four-cycle* from left to right in three steps. Each yellow node corresponds to an XOR on the variables of incident edge labels. First, all nodes in the cycle are *split* inward with XOR factorization. Then, neighboring nodes in the inner cycle are *merged* by variable elimination. Finally, the cycle is *collapsed* by variable elimination.

46

4 Experiments

The experiments were run on the bwForCluster Helix with AMD Milan EPYC 7513 CPUs. we enforced a memory limit of 7 GB and a time limit of 5000 s per solver run. We present results on four different Tseitin families, that differ in their underlying graph structure.

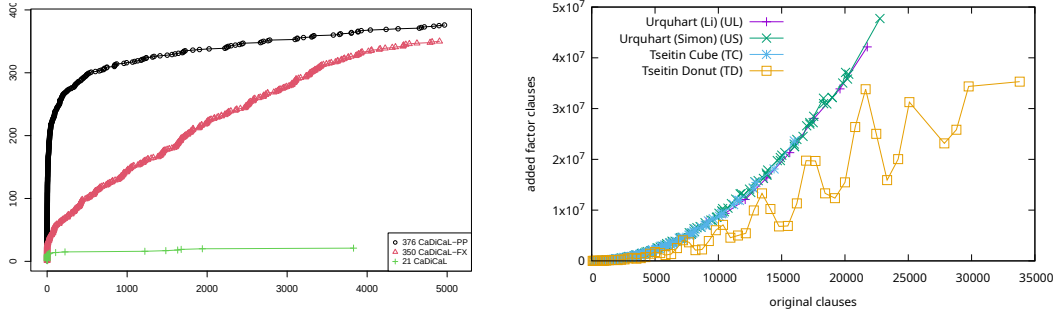


Figure 2 (a) The CADICAL-PP solver outperforms CADICAL-FX while CADICAL times out on all but the smallest instances. (b) We observe a clear quadratic dependency in the number of *split* operations (cf. Sect. 3) and initial formula size, for all but the Tseitin Donuts, which have linear size proofs in our *split-and-merge* paradigm.

50

5 Conclusion

This paper presents advanced gate factorization, an extension to BVA that relies on learned clauses to reencode XOR and ITE gates during inprocessing. We show how, with slight modifications, XOR factorization can be used in preprocessing to generate polynomial proofs for Tseitin formulas. Our experiments show the effectiveness of both approaches on Tseitin formulas and empirically confirm that our solver is generating polynomial proofs.

56

References

- 1 Gilles Audemard and Laurent Simon. On the Glucose SAT solver. *Int. J. Artif. Intell. Tools*, 27(1):1840001:1–1840001:25, 2018. doi:10.1142/S0218213018400018.
- 2 Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors. *Handbook of Satisfiability - Second Edition*, volume 336 of *Frontiers in Artificial Intelligence and Applications*. IOS Press, 2021. doi:10.3233/FAIA336.
- 3 Martin Davis and Hilary Putnam. A computing procedure for quantification theory. *J. ACM*, 7(3):201–215, 1960. doi:10.1145/321033.321034.
- 4 Niklas Eén and Armin Biere. Effective preprocessing in SAT through variable and clause elimination. In Fahiem Bacchus and Toby Walsh, editors, *Theory and Applications of Satisfiability Testing*, pages 61–75, Berlin, Heidelberg, 2005. Springer Berlin Heidelberg.
- 5 Matti Järvisalo, Marijn J. H. Heule, and Armin Biere. Inprocessing rules. In Bernhard Gramlich, Dale Miller, and Uli Sattler, editors, *Automated Reasoning*, pages 355–370, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- 6 Norbert Manthey, Marijn Heule, and Armin Biere. Automated reencoding of Boolean formulas. In Armin Biere, Amir Nahir, and Tanja E. J. Vos, editors, *Hardware and Software: Verification and Testing - 8th International Haifa Verification Conference, HVC 2012, Haifa, Israel, November 6-8, 2012. Revised Selected Papers*, volume 7857 of *Lecture Notes in Computer Science*, pages 102–117. Springer, 2012. doi:10.1007/978-3-642-39611-3_14.
- 7 João Marques-Silva, Inês Lynce, and Sharad Malik. Conflict-driven clause learning SAT solvers. In Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh, editors, *Handbook of Satisfiability*, volume 336 of *Frontiers in Artificial Intelligence and Applications*, pages 133–182. IOS Press, 2nd edition, 2021. doi:10.3233/FAIA200987.
- 8 Alexander Nadel and Vadim Ryvchin. Chronological backtracking. In Olaf Beyersdorff and Christoph M. Wintersteiger, editors, *Theory and Applications of Satisfiability Testing - SAT 2018 - 21st International Conference, SAT 2018, Held as Part of the Federated Logic Conference, FloC 2018, Oxford, UK, July 9-12, 2018, Proceedings*, volume 10929 of *Lecture Notes in Computer Science*, pages 111–121. Springer, 2018. doi:10.1007/978-3-319-94144-8_7.
- 9 Chanseok Oh. Between SAT and UNSAT: the fundamental difference in CDCL SAT. In Marijn Heule and Sean A. Weaver, editors, *Theory and Applications of Satisfiability Testing - SAT 2015 - 18th International Conference, Austin, TX, USA, September 24-27, 2015, Proceedings*, volume 9340 of *Lecture Notes in Computer Science*, pages 307–323. Springer, 2015. doi:10.1007/978-3-319-24318-4_23.